

Υπολογιστική Γεωμετρία και Μιγαδική Ανάλυση

Βέλτιστοι Αλγόριθμοι για κυρτές θήκες

Γομάτος Ηλίας

AM 1112 2022 00026

Περίληψη

Σε αυτή την εργασία διερευνούμε μία εικασία η οποία διατυπώθηκε στο μάθημα της μιγαδικής ανάλυσης. Σκοπός είναι η μείωση του χρόνου σε επίπεδο ασυμπτωτικής ανάλυσης. Αντί για λεπτομερείς αποδείξεις, δίνονται περιγραφικοί αλγόριθμοι μαζί με ψευδοκώδικα για την υλοποίησή τους. Η εργασία χωρίζεται σε δύο μέρη, στην εύρεση ριζών που παρουσιάζεται πολύ περιληπτικά και στα κομμάτια υπολογιστικής γεωμετρίας.

Περιεχόμενα

1	Εισαγωγή	4
1.1	Η Εικασία	4
1.2	Τα επιμέρους κομμάτια	4
2	Εύρεση Ριζών	5
2.1	Η εύρεση του κύκλου	5
2.2	Υπολογισμός των Συντελεστών	6
2.3	Συνολική πολυπλοκότητα	6
3	Υπολογιστική γεωμετρία	7
3.1	Υπολογισμός Κυρτής Θήκης	7
3.2	Τομή Κυρτών Πολυγώνων	9
3.2.1	Ο Αλγόριθμος του O'Rourke	9
3.2.2	Περαιτέρω μέλετη πολυπλοκότητας	13
3.3	Απόσταση Hausdorff	13
4	Συμπεράσματα	16

1 Εισαγωγή

Έστω ένα πολυώνυμο $P(z)$ και την παράγωγό του $P'(z)$. Ορίζουμε ως G_0 και G' τις κυρτές θήκες των ριζών τους αντίστοιχα. Από τη θεωρία της μιγαδικής Ανάλυσης (Πόρισμα Lukas) γνωρίζουμε ότι $G' \subset G_0$. Έστω μία αυθαίρετη σταθερά c ορίζουμε $P_c(z) = P(z) + c$ και ορίζουμε G_c όπως πριν. Παρατηρούμε ότι $G' \subset G_c$, άρα $G' \subset [G_0 \cap G_c]$.

1.1 Η Εικασία

Γενικά μπορούμε να διαλέξουμε αυθαίρετα πολλά ακόμη c και να ισχύει ο εγκλεισμός. Δηλαδή ισχύει ότι $G' \subset \bigcap_{c \in \mathbb{C}} G_c$.

Θέλουμε να δούμε εαν ισχύει:

$$G' = \bigcap_{c \in \mathbb{C}} G_c$$

Για συντομία θέτουμε $\bigcap_{c \in \mathbb{C}} G_c = G$

1.2 Τα επιμέρους κομμάτια

Για να μοντελοποιήσουμε αυτό το πρόβλημα αλγοριθμικά χρειαζόμαστε να μπορούμε να υπολογίσουμε τα G και G' . Για να προσεγγίζουμε το G θα διαλέξουμε διάφορες τιμές για το c . Άρα πρέπει να μπορούμε να υπολογίσουμε τις ρίζες ενός πολυωνύμου, την κυρτή θήκη των ριζών και την τομή δύο κυρτών συνόλων. Επίσης για να επιβεβαιώσουμε πιο μαθηματικά την εικασία θα υπολογίσουμε την απόσταση Hausdorff των δύο συνόλων, η οποία αναμένουμε να συγχλίνει στο 0.

Για το υπόλοιπο της εργασίας όπου θεωρούμε n εννοείται ως ο βαθμός του πολυωνύμου που έχουμε σταθεροποιήσει στην αρχή, και k το πλήθος των c που έχουμε επιλέξει (Δηλαδή των επιπλέον πολυωνύμων) για να προσεγγίσουμε το G . Τέλος δεν θα ασχοληθούμε με το μέγεθος των αριθμών που διαχειριζόμαστε, υποθέτουμε ότι όλες οι πράξεις καθώς και τυχών δεσμεύσεις μνήμης, γίνονται στιγμιαία.

2 Εύρεση Ριζών

Ο αλγόριθμος αυτός δόθηκε από τον Victor Pan το 1995 και αποτελεί αποδεδειγμένα τον πιο γρήγορο τρόπο υπολογισμού ριζών. Η Γενική ιδέα βασίζεται σε Divide and Conquer διαιρώντας στο πολυώνυμο κάθε φορά στα δύο. Για ένα πολυώνυμο P ψάχνουμε καινούργια πολυώνυμα P_1, P_2 τέτοια ώστε $P = P_1 P_2$. Αυτή η διαδικασία θα χρειαστεί να γίνει $\log(n)$ φορές. Σκοπός είναι καταλήξουμε σε πολυώνυμο πρώτου βαθμού, όπου εμφανώς υπολογίζονται άμεσα οι ρίζες τους.

Για να γίνει η διαίρεση αυτή θα πάρουμε ένα δίσκο με κέντρο το 0 όπου θα χωρίζει τις ρίζες του πολυωνύμου στα δύο, τις μισές μέσα στο δίσκο και τις μισές έξω. Αυτός ο δίσκος θα βρεθεί με τη μέθοδο του Graeffe. Έπειτα θα βρούμε μία σχέση για το άθροισμα των ριζών και μέσω σχέσεων του Νεύτωνα μπορούμε να υπολογίσουμε τους συντελεστές του πολυωνύμου με τις ρίζες μέσα στο δίσκο. Τέλος κάνοντας τη διαίρεση μπορούμε να βρούμε και το πολυώνυμο με τις ρίζες έξω από το δίσκο.

Το πολυώνυμο με τις ρίζες μέσα στο δίσκο θα το λέμε P_{in} και αυτό με τις ρίζες έξω P_{out} .

2.1 Η εύρεση του κύκλου

Αρχικά για να βρούμε τον κύκλο θα εφαρμόσουμε τη μέθοδο του Graeffe για να μεγάλώσουμε τις αποστάσεις μεταξύ των ριζών και να γίνει πιο ξεκάθαρη η εύρεση του κύκλου. Συγκεκριμένα θα ορίζουμε:

$$P_{new} = (-1)^n P(-\sqrt{z}) P(\sqrt{z})$$

Παρατηρούμε το P_{new} είναι ένα πολυώνυμο n -οστού βαθμού και ότι οι ρίζες του P_{new} θα είναι απλώς οι ρίζες του P στο τετράγωνο. Συνεχίζοντας αυτή τη διαδικασία μπορούμε να μεγάλώσουμε κατά πάρα πολύ τις αποστάσεις των ριζών.

Για να προσδιορίσουμε την ακτίνα του κύκλου θα φτιάξουμε την άνω κυρτή θήκη των σημείων $(i, \log(|a_i|))$ και από θεώρημα σειρών Puiseux, μπορούμε να βρούμε το που κινείται απότομα η κλίση του συνόλου, που είναι το σημείο ακριβώς που ψάχνουμε καθώς εκεί θα υπάρχει η μεγαλύτερη διαφορά των ριζών στο πολυώνυμο.

Δύο ειδικές περιπτώσεις είναι εάν το πολυώνυμο έχει ρίζες που ισαπέχουν από το 0 και αν έχει ρίζες με μεγάλη πολλαπλότητα. Στην πρώτη περίπτωση μπορούμε απλώς να θεωρήσουμε το πολυώνυμο $P(x - x_0)$ για κάποιο μεγάλο x_0 , έτσι αποφεύγεται το πρόβλημα αλλά και αντιστοιχούν εύκολα οι ρίζες στο αρχικό. Στην δεύτερη περίπτωση εάν κάνουμε τη μετακίνηση και έχουμε το ίδιο πρόβλημα τότε καταλαβαίνουμε ότι έχουμε ρίζες με πολλαπλότητα. Τότε απλώς μπορούμε να χωρίσουμε το πολυώνυμο στις ρίζες της πολλαπλότητας και να υπολογίσουμε κατευθείαν τις ρίζες αυτές.

Η εύρεση του νέου πολυωνύμου θέλει $O(n \log(n))$ χρόνο και ομοίως για την άνω κυρτή θήκη. Άρα συνολικά $O(n \log(n))$ για αυτό το μέρος.

2.2 Υπολογισμός των Συντελεστών

Εφόσον έχουμε βρει το κύκλο, θα χρησιμοποιήσουμε το εξής ολοκλήρωμα για να προσδιορίσουμε τις ρίζες.

$$\frac{1}{2\pi i} \int_C z^k \frac{P'(z)}{P(z)} dz = \sum_{i \in I} r_i^k = s_k$$

Όπου r_i οι ρίζες του πολωνύμου. Η ισότητα προκύπτει άμεσα από τον ολοκληρωτικό τύπο του Cauchy.

Επίσης γνωρίζουμε τον εξής τύπο του Νεύτωνα

$$\forall k, k a_k + \sum_{j=1}^k s_j a_{k-j} = 0$$

Όπου a_k οι συντελεστές του πολωνύμου. Άρα μπορούμε αναδρομικά να υπολογίσουμε τους συντελεστές του πολωνύμου. Μέσω FFT και της μεθόδου Newton-Raphson μπορούν να υπολογιστούν σε $O(n \log(n))$ χρόνο αντί για τετραγωνικό. Έτσι βρίσκουμε το πολώνυμο P_{in} .

Γνωρίζουμε ότι μπορούμε να κάνουμε διαίρεση πολωνύμων σε $O(n \log(n))$, οπότε ο υπολογισμός του $P_{out} = \frac{P}{P_{in}}$ μπορεί να γίνει σε τόσο χρόνο.

2.3 Συνολική πολυπλοκότητα

Η συνολική πολυπλοκότητα των ριζών βγαίνει $O(n \log^2(n))$ για να υπολογίσουμε τις ρίζες, αφού θα χρειαστεί να κάνουμε τις παραπάνω διαδικασίες $\log(n)$ φορές. Όμως εφόσον θα χρειαστεί να το κάνουμε για κάθε πολώνυμο που θα ορίσουμε τότε η πολυπλοκότητα γίνεται $O(kn \log(n))$.

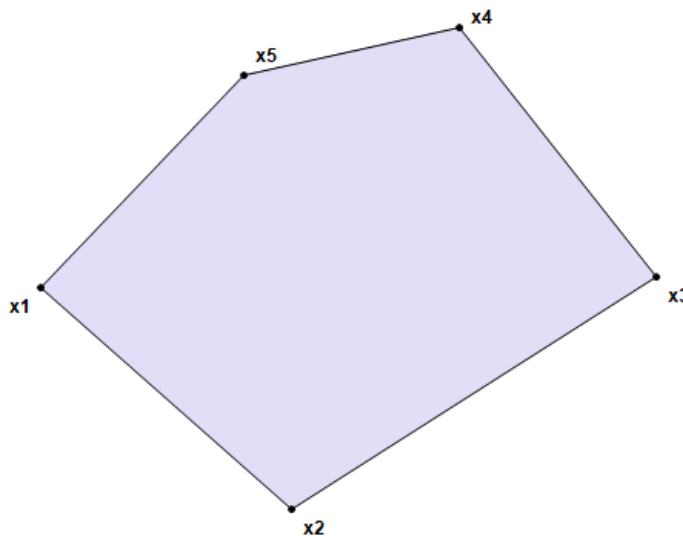
Περισσότερη έμφαση δίνεται παρακάτω για τα υπόλοιπα κομμάτια του αλγορίθμου καθώς η εύρεση ριζών αποτελεί κυρίως άλλους κλάδους μαθηματικών. Για περισσότερες λεπτομέρειες δείτε το [1]

3 Υπολογιστική γεωμετρία

Εφόσον έχουμε θέσει ένα αρχικό άνω φράγμα για τον υπολογισμό των ριζών, θα χρησιμοποιήσουμε αποτελέσματα της υπολογιστικής γεωμετρίας για να παραμείνουμε όσο πιο κοντά του γίνεται. Γενικά μία αφελής μέθοδος (brute force) θα χρειαζόταν να κοιτάει πως αλληλεπιδρά κάθε ακραίο σημείο με κάθε άλλο, που θα οδηγούσε σε πολυπλοκότητα $O(k^2n^2)$, μπορούμε όμως να κάνουμε κάτι καλύτερο!

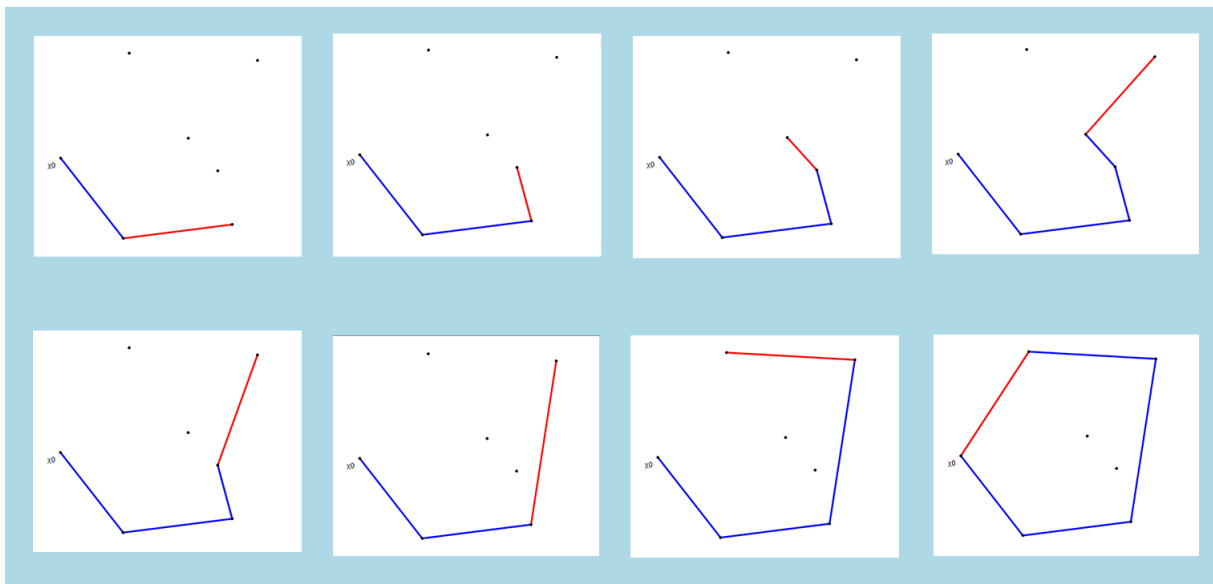
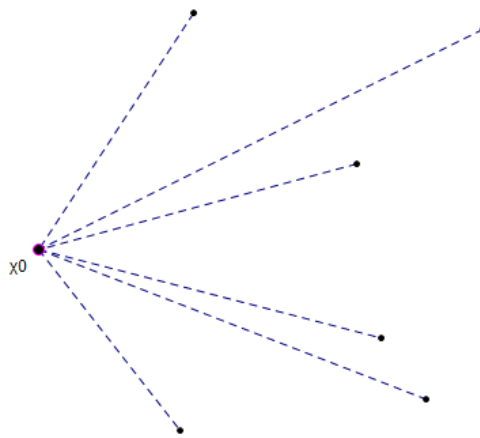
3.1 Υπολογισμός Κυρτής Θήκης

Πλέον έχουμε τις ρίζες του πολυωνύμου σε μία λίστα και θέλουμε να βρούμε την κυρτή τους θήκη. Για αρχή πρέπει να βρούμε έναν τρόπο να αναπαριστάμε κυρτές θήκες. Από την θεωρία γνωρίζουμε ότι ένα κυρτό σύνολο περιγράφεται πλήρως από τα ακραία του σημεία, τα οποία στη περίπτωση του προβλήματος είναι πεπερασμένα. Άρα θα μπορούσαμε απλά να αποθηκεύουμε (εφόσον τα βρούμε) τα ακραία σημεία του συνόλου των ριζών, με τυχαία σειρά. Όμως διαισθητικά (για να είναι σχήμα) αλλά και υπολογιστικά θα προτιμούσαμε να τα έχουμε “σε σειρά” δηλαδή αριστερόστροφα, όπως δηλαδή θα τα είχαμε εάν θέλαμε να σχεδιάσουμε την κυρτή θήκη στο χαρτί. Έτσι από εδώ και πέρα όταν λέμε κυρτή θήκη εννοούμε μια λίστα των ακραίων σημείων αριστερόστροφα από το κέντρο του συνόλου. Για παράδειγμα, για το σχήμα μια αναπαράσταση θα ήταν η λίστα $[x1, x2, x3, x4, x5]$, ή η $[x3, x4, x5, x1, x2]$.



Ο αλγόριθμος που θα δούμε δόθηκε από τον Ronald Graham το 1972. Η γενική ιδέα του αλγορίθμου είναι η εξής: θέλουμε να κινούμαστε στη περίμετρο του συνόλου κυκλικά, μέχρι να το κυκλώσουμε όλο. Υποθέτοντας ότι είμαστε στη περίμετρο πάμε από κορυφή σε κορυφή κυκλικά, εάν πρέπει να “μπούμε μέσα” και μετά να ξαναβγούμε πάλι έξω τότε άσκοπα μπήκαμε και μπορούμε απλώς να βγάλουμε τις κορυφές στις οποίες μπαιναμε μέσα. Με το τρόπο αυτό χτίζουμε σιγά σιγά τη περίμετρο του συνόλου, το οποίο είναι ακριβώς η κυρτή θήκη.

Άρα πρέπει να ξεκινήσουμε στην περίμετρο και να παραμείνουμε εκεί. Το πρώτο κομμάτι είναι εύκολο, αρκεί να ξεκινήσουμε από την κορυφή με τη μικρότερη τετμημένη, έστω x_0 , καθώς σίγουρα θα είναι ακραίο σημείο του συνόλου. Για να συνεχίσουμε να κινούμαστε στη περίμετρο, θα βάλουμε σε μια καινούργια λίστα την γωνία κάθε άλλου σημείου με το x_0 ως προς τον άξονα x . Έπειτα ταξινομήσουμε τη λίστα αυτή και θα κοιτάμε τα σημεία με τη σειρά στη λίστα. Θα κινούμαστε με τη σειρά της λίστας αυτής, αν κινηθούμε προς τα αριστερά τότε όλα πάνε καλά, αλλιώς μπήκαμε μέσα στο σχήμα και ξαναβγαίνουμε. Σε αυτή την περίπτωση θα κάνουμε backtrack και αφαιρούμε την κορυφή που μας έβαλε μέσα. Στο πρώτο σχήμα βλέπουμε πως θα μετρούσαμε αρχικά τις κλήσεις, και στο δεύτερο πως φτιάχνεται σιγά σιγά η κυρτή θήκη.



Για να υλοποιήσουμε αυτή τη διαδικασία θα χρησιμοποιήσουμε μια στοίβα. Η κορυφή της στοίβας μας επιτρέπει να ξέρουμε που βρισκόμαστε και για να κάνουμε backtrack αρκεί να αφαιρέσουμε το δεύτερο πιο πάνω στοιχείο της στοίβας. Ένας πιο αυστηρός ψευδοκώδικας είναι ο παρακάτω. Περισσότερες λεπτομέρειες στο [2]

Algorithm 1 Αλγόριθμος Εύρεσης Κυρτής Θήκης

```
1:  $x_0 = \text{Find-Low-}y(A)$  ▷ Βρίσκουμε το  $x_0$  με τη μικρότερη τετμημένη
2:  $B = \text{empty.list}()$  ▷ Ο πίνακας  $A$  έχει τις ρίζες του πολυωνύμου
3: for  $i \leftarrow 1$  to  $n$  do
4:    $B[i] = (\frac{\text{Im}(A[i]) - \text{Im}(x_0)}{\text{Re}(A[i]) - \text{Re}(x_0)}, A[i])$ 
5: end for
6:  $\text{Sort}(B)$ 
7:  $s = \text{empty.stack}()$ 
8:  $s.\text{push}(x_0)$ 
9:  $s.\text{push}(B[1].\text{second})$ 
10: for  $i \leftarrow 0$  to  $n$  do
11:    $\text{top} = s.\text{top}()$  ▷ Το πάνω πάνω στοιχείο της στοίβας
12:    $\text{top2} = s.\text{top2}()$  ▷ Το δευτερο πάνω πάνω στοιχείο της στοίβας
13:   while  $\text{outprod}(B[i].\text{second} - \text{top2}, \text{top} - B[i].\text{second}) \leq 0$  do
14:      $s.\text{pop}()$  ▷ outprod είναι το εξωτερικό γινόμενο
15:      $\text{top} = s.\text{top}()$ 
16:      $\text{top2} = s.\text{top2}()$ 
17:   end while
18:    $s.\text{push}(B[i].\text{second})$ 
19: end for
20: return  $s$ 
```

Για τον αλγόριθμο παρατηρούμε ότι, οι αρχικοποιήσεις των B, s θέλουν σταθερό χρόνο, το πρώτο *for* εκτελεί μόνο πράξεις, οπότε θέλει γραμμικό χρόνο. Στο δεύτερο οι πράξεις $\text{top}, \text{top2}$ εκτελούνται σε σταθερό χρόνο και για το *while* στη στοίβα μπορεί να μπει το κάθε σημείο το πολύ μία φορά, άρα θα χρειαστούμε το πολύ γραμμικό χρόνο και για αυτό το *for*. Τέλος το *sort* θέλει $O(n \log n)$ χρόνο, οπότε συνολικά ο αλγόριθμος αυτός θέλει $O(n \log n)$ χρόνο.

Άρα για k πολυώνυμα χρειαζόμαστε $O(kn \log n)$ χρόνο, το οποίο είναι το καλύτερο που θα μπορούσαμε να κάνουμε ¹.

3.2 Τομή Κυρτών Πολυγώνων

Πλέον έχουμε στα χέρια μας τις κυρτές θήκες, για να υπολογίσουμε όμως το G πρέπει να μπορούμε να βρούμε και την τομή τους. Ο Αλγόριθμος παρακάτω δόθηκε από τον O'Rourke το 1982 και μας επιτρέπει να βρίσκουμε τομές 2 κυρτών συνόλων (με n_1, n_2 κορυφές) σε χρόνο $O(n_1 + n_2)$. Εδώ το k θα παίζει περισσότερο ρόλο από πριν καθώς έχει σημασία με ποιά σειρά γίνεται η τομή, αφού τα n_1, n_2 θα είναι συναρτήσεις και του k .

3.2.1 Ο Αλγόριθμος του O'Rourke

Η γενική ιδέα για την εύρεση της τομής βασίζεται στο ότι για να φτιάξουμε την τομή αρκεί να κινούμαστε στη περίμετρο των σχημάτων ταυτόχρονα έχοντας καθορίσει ποιο κομμάτι είναι

¹Εάν μπορούσαμε να υπολογίσουμε κυρτές θήκες σε χρόνο καλύτερο από αυτό, τότε θα μπορούσαμε να κάνουμε και ταξινόμηση σε κάτι καλύτερο!

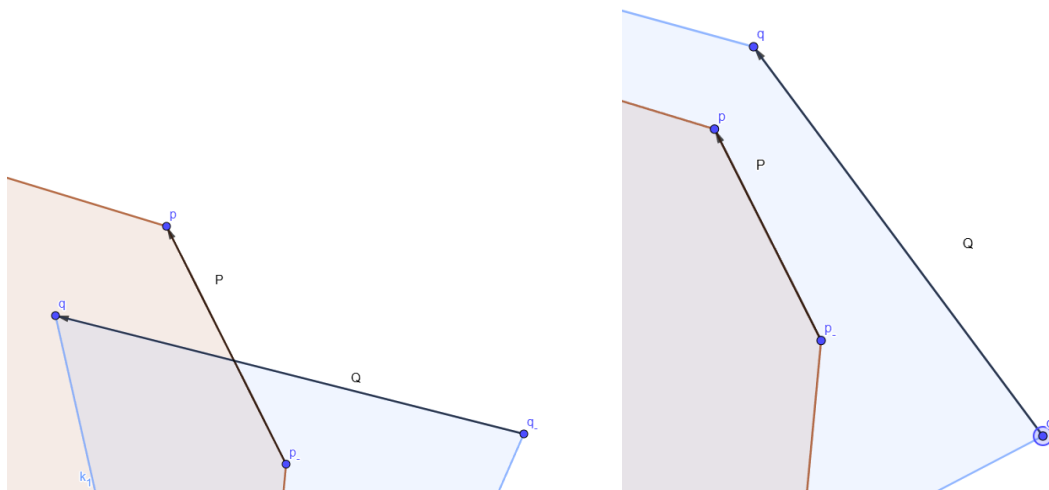
μέσα και ποιό έξω. Όμως έχουμε στη διαθεσή μας μόνο τα ακραία σημεία των συνόλων, άρα πρέπει να κινούμαστε πάνω στις κορυφές αυτές, για την ακρίβεια θα κινούμαστε στις πλευρές τους. Για να κινηθούμε κυκλικά αρκεί να έχουμε δύο δείκτες όπου ο καθένας ακολουθεί το ένα από τα δύο σχήματα και να προχωράει ο καθένας κατάλληλα με κάποιο κανόνα. Αυτός ο κανόνας είναι το δύσκολο κομμάτι του αλγορίθμου. Η τομή στο τέλος θα είναι όλα τα σημεία στα οποία προχωρήσε ο μέσα δείκτης.

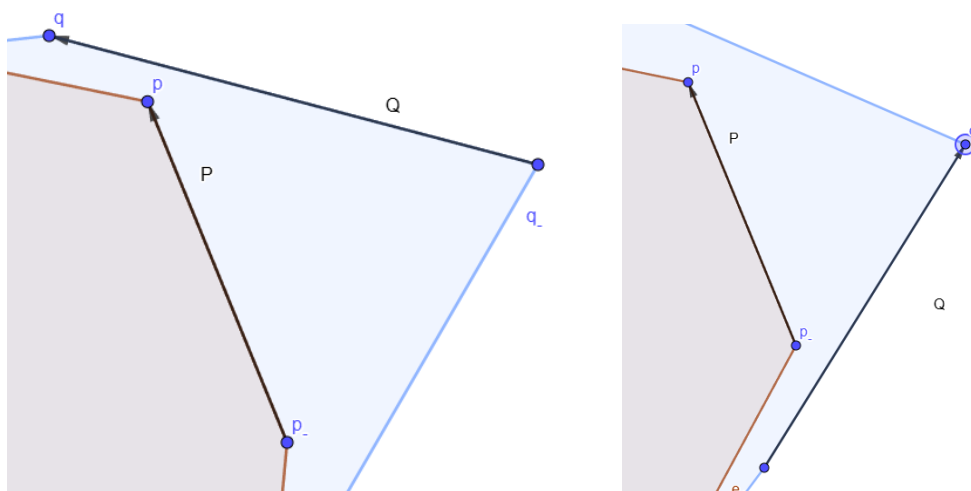
Για τους δύο δείκτες θα θεωρούμε δύο σημεία p, q και τα προηγούμενα τους σημεία p_-, q_- και σαν διανύσματα $\mathbf{p}, \mathbf{q}$ αυτά που ξεκινάνε από το προηγούμενο προς το επόμενο σημείο. Επίσης θα χρησιμοποιούμε την έννοια του αριστερά ενός διανύσματος, που μαθηματικά σημαίνει ότι βρίσκεται στο αριστερό ημιεπίπεδο που σχηματίζει η διεύθυνση του διανύσματος.

Ο αλγόριθμος χωρίζεται σε δύο φάσεις. Πρέπει να αρχικοποιήσουμε τους δείκτες και να βρούμε ποιός είναι μέσα και ποιός έξω και αφού το κάνουμε αυτό μπορούμε να αρχίσουμε να κρατάμε τα μέσα σημεία για να σχηματιστεί η τομή. Για την αρχικοποίηση αρκεί να χρησιμοποιήσουμε θα διαλέξουμε 2 τυχαία p, q και θα ακολουθούμε τον κανόνα που θα θέσουμε μέχρι να τέμνονται τα $\mathbf{p}, \mathbf{q}$. Αν ξαναγυρίσουν στην θέση τους και δεν τέμνονται τότε το ένα βρίσκεται ολόκληρο μέσα στο άλλο, άρα η τομή θα είναι απλά το ένα από τα δύο.

Ο κανόνας που θα χρησιμοποιήσουμε χωρίζεται στις περιπτώσεις που φαίνονται στον πίνακα και στις εικόνες.

Περίπτωση	Κατάσταση	Αποτέλεσμα
α	Τα $\mathbf{p}, \mathbf{q}$ τέμνονται	προχωράμε και τα δύο και εναλλάσσουμε τα μέσα και έξω
β	Το έξω βρίσκεται εξ ολοκλήρου δεξιά του μέσα	προχωράει το έξω
γ	Τα έξω και μέσα τέμνονται μετά το τέλος του μέσα	προχωράει το μέσα
δ	Τα έξω και μέσα τέμνονται πριν το τέλος του μέσα	προχωράει το έξω





Στα σχήματα από πάνω φαίνονται οι περιπτώσεις απο αριστερά προς τα δεξιά όπως αναγράφονται στον πίνακα. Ακολουθεί ένας ψευδοκώδικας παρακάτω. Για περισσότερες λεπτομέρειες δείτε [3]

Algorithm 2 Αλγόριθμος Εύρεσης τομής

```
1:  $p \leftarrow \text{random}(P_1)$  ▷ Διαλεγουμε μία τυχαία πλευρά
2:  $q \leftarrow \text{random}(P_2)$  ▷ από κάθε πολύγωνο
3: while p and q dont intersect do ▷ Αρχικά πρέπει να βρούμε που τέμνονται
4:   if  $\text{outprod}(p, q) > 0$  then ▷ για να αρχικοποιήσουμε τα μέσα και έξω
5:     if  $p \in hp(q)$  then
6:       advance  $q$ 
7:     else
8:       advance  $p$ 
9:     end if
10:  else
11:    if  $q \in hp(p)$  then
12:      advance  $p$ 
13:    else
14:      advance  $q$ 
15:    end if
16:  end if
17: end while
18: if  $\text{outprod}(p, q) > 0$  then
19:    $p \leftarrow \text{inside}$ 
20:    $q \leftarrow \text{outside}$ 
21: else
22:    $q \leftarrow \text{inside}$ 
23:    $p \leftarrow \text{outside}$ 
24: end if
25:  $s = \text{empty.Stack}()$ 
26: while p or q distinct do ▷ Παρόμοιο με το προηγούμενο while απλά τώρα
27:   if  $\text{outprod}(p, q) > 0$  then ▷ τα μέσα. Σταματάμε όταν δούμε ίδια p,q
28:     if  $p \in hp(q)$  then
29:       if  $q = \text{inside}$  then
30:          $s.\text{push}(q)$ 
31:       end if
32:       advance  $q$ 
33:     else
34:       if  $p = \text{inside}$  then
35:          $s.\text{push}(p)$ 
36:       end if
37:       advance  $p$ 
38:     end if
39:   else
40:     if  $q \in hp(p)$  then
41:       if  $p = \text{inside}$  then
42:          $s.\text{push}(p)$ 
43:       end if
44:       advance  $p$ 
45:     else
46:       if  $q = \text{inside}$  then
47:          $s.\text{push}(q)$ 
48:       end if
49:       advance  $q$ 
50:     end if
51:   end if
52: end while
```

3.2.2 Περαιτέρω μέλετη πολυπλοκότητας

Για τον αλγόριθμο που δόθηκε, παραλείψαμε την περίπτωση του τι γίνεται αν τα p, q δεν συναντιθούν ποτέ εξ αρχής (στο πρώτο `while`). Τότε σημαίνει ότι το ένα βρίσκεται εξ ολοκλήρου μέσα στο άλλο. Άρα αρκεί να παρθεί ένα σημείο και να δούμε εάν βρίσκεται μέσα στο άλλο. Αυτή η περίπτωση δε χαλάει την πολυπλοκότητα καθώς αυτό μπορεί να γίνει σε λογαριθμικό χρόνο.

Αν υποθέσουμε ότι δεν έχουμε αυτή την περίπτωση, τότε τα p, q θα κάνουν το πολύ τον γύρο της περιμέτρου όλου του σχήματος μέχρι να βρεθούν, άρα το πρώτο `while` έχει πολυπλοκότητα $O(n_1 + n_2)$ όπου n_1, n_2 το πλήθος των κορυφών των πολυγώνων. Ομοίως για το δεύτερο `while` αφού θα κάνουν πάλι το πολύ έναν γύρο και μέσα στο `while` υπάρχουν μόνο πράξεις. Άρα συνολικά ο αλγόριθμος έχει πολυπλοκότητα $O(n_1 + n_2)$.

Όπως προαναφέρθηκε, η σειρά με την οποία θα γίνουν οι τομές έχει σημασία. Εάν υποθέσουμε την χειρότερη περίπτωση, δηλαδή ότι πάντα η τομή δυο πολυγώνων οδηγεί σε ένα νέο πολύγωνο με πλήθος κορυφών ίσο με το άθροισμα των κορυφών δύο πολυγώνων, τότε εάν κάναμε τις τομές με τη σειρά θα είχαμε να κάνουμε τόσες πράξεις:

$$(n + n) + (n + 2n) + \dots + (n + (k - 1)n) = (k - 1)n + n \frac{k(k - 1)}{2}$$

άρα πολυπλοκότητα $O(k^2 n)$. Εάν όμως αντί αυτού κάναμε τις τομές από τις μικρότερες προς τις μεγαλύτερες (Divide and Conquer), δηλαδή πρώτα όλα τα πολύγωνα με n κορυφές και μετά όλες με $2n$ κτλπ θα είχαμε να κάνουμε:

$$\frac{k}{2} 2n + \frac{k}{4} 4n + \dots + \frac{k}{k} kn = nk \log k$$

Άρα πολυπλοκότητα $O(nk \log(k))$ που είναι αρκετά καλύτερο.

Άρα συνολικά αυτό το κομμάτι του αλγορίθμου παίρνει $O(nk \log(k))$ χρόνο.

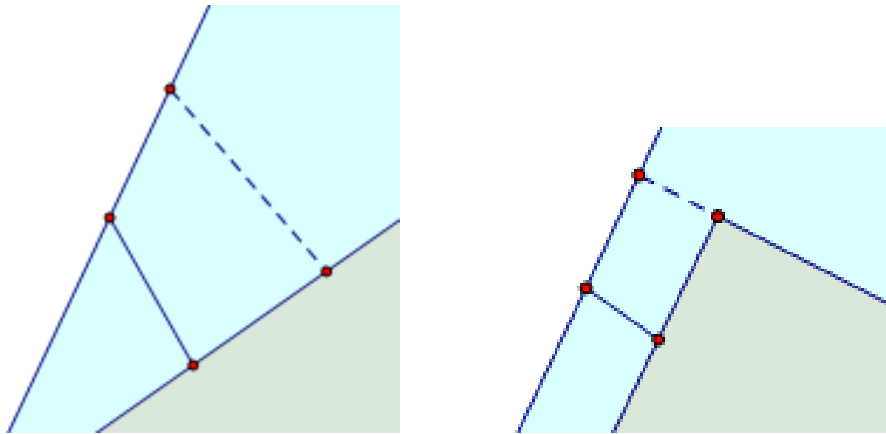
3.3 Απόσταση Hausdorff

Οι παραπάνω διαδικασίες θα ήταν επαρκής για μια οπτικοποίηση της λύσης. Όμως εάν θέλαμε μαθηματικά να επιβεβαιώσουμε αυτό που βλέπουμε, θα έπρεπε να κοιτάζουμε το πόσο απέχουν τα σχήματα μεταξύ τους. Για την ακρίβεια θα έπρεπε να δούμε πόσο απέχει το μακρινότερο σημείο του G από το G' . Αυτή η απόσταση λέγεται απόσταση Hausdorff. Πιο συγκεκριμένα ορίζεται ως:

$$\text{Hausdorff}(G, G') = \max_{x \in G} [\min_{y \in G'} \text{dist}(x, y)]$$

Εμείς θα θέλαμε η απόσταση αυτή να είναι μικρή. Για να την υπολογίσουμε πρώτα θα κάνουμε τις εξής παρατηρήσεις. Αρχικά εάν η μέγιστη απόσταση ήταν μεταξύ δύο όχι ακραίων σημείων, τότε θα πρέπει οι πλευρές αυτών των σημείων να είναι παράλληλες. Αν δεν ήταν θα μπορούσαμε να διαλέξουμε κάποια άλλα σημεία στα πιο ανοιχτά που να απείχαν περισσότερο, όπως φαίνεται στη φωτογραφία (Εικόνα 1, Το ευθύγραμμο τμήμα είναι η αρχική

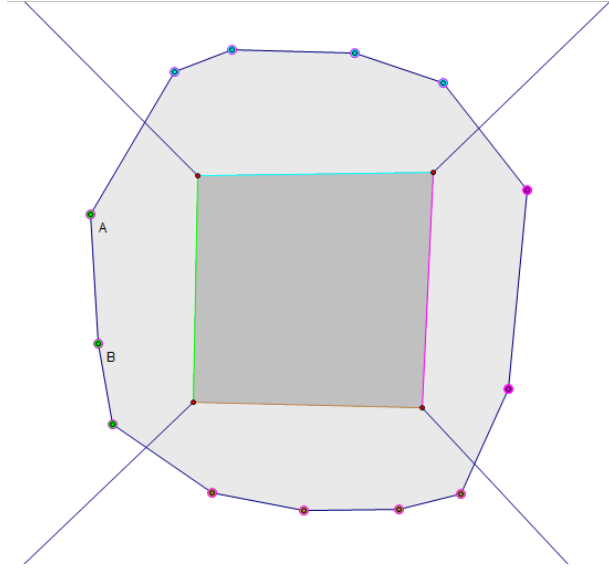
μας υπόθεση για τη μέγιστη απόσταση, με διακεκομμένες η νέα μεγαλύτερη απόσταση).



Δεύτερη παρατήρηση είναι ότι η μέγιστη απόσταση μπορεί να είναι σε μία από τις κορυφές ενός από τα πολύγωνα. Εάν η απόσταση προέρχεται από παράλληλες ακμές, τότε μπορούμε να κινήσουμε την ευθεία μέχρι να πετύχουμε μία κορυφή (Εικόνα 2). Τέλος παρατηρούμε και ότι η απόσταση θα μεγιστοποιείται σε κάποια από τις κορυφές του G .

Άρα αρκεί να βρούμε την μέγιστη απόσταση, από τις κορυφές του G προς το G' . Για να το κάνουμε αυτό πρώτα θα βρούμε σε ποιά ακμή ανήκει κάθε κορυφή και έπειτα θα βρούμε την απόσταση της.

Ας ξεκινήσουμε παίρνοντας μία τυχαία κορυφή του G . Θα βρούμε την κοντινότερη ακμή του ως προς το G' . Τώρα παρατηρούμε ότι οι υπόλοιπες κορυφές που θα ανήκουν στην ίδια ακμή θα είναι κάποιες συνεχόμενες από τα αριστερά και τα δεξιά, συγκεκριμένα αυτές που θα ανήκουν στο χωρίο που ανήκουν που ορίζουν οι εξωτερικοί διχοτόμοι από την ακμή που τη συνδέει. Έτσι μπορούμε να αντιστοιχίσουμε όλες τις κορυφές της ακμής αυτή κάνοντας binary search βρίσκοντας που ξεκινάει και που τελειώνει η αντιστοίχιση της κάθε πλευράς. Μια τέτοια αντιστοίχιση φαίνεται στο σχήμα (Οι μπλε ακμές αντιστοιχούν στην μπλε πλευρά κλπ).



Ένας ψευδοαλγόριθμος για τη διαδικασία αυτή φαίνεται παρακάτω.

Algorithm 3 Αλγόριθμος Εύρεσης Απόστασης Hausdorff

```

1:  $x = \text{random}(G)$                                 ▷ Διαλέγουμε μία τυχαία κορυφή για να αρχίσουμε
2:  $e = \text{find}(x)$                                     ▷ Βρισκουμε brute force τη κοντινότερη ακμή
3:  $\text{assigned}(x) \leftarrow e$ 
4:  $\text{BinSear}(e)$                                     ▷ Βρισκουμε όλες τις κορυφές της  $e$  με δυαδική αναζήτηση
5: while  $(\exists x \in G) \text{assigned}(x) = \text{None}$  do
6:    $x = \text{first}_{unassigned}()$ 
7:    $\text{assigned}(x) = \text{assigned}(x_-) + 1$ 
8:    $e_2 = \text{assigned}(x)$ 
9:    $\text{BinSear}(e_2)$ 
10: end while

```

Ο Αλγόριθμος αυτός απαιτεί $O(n)$ για την εύρεση της πρώτης ακμής, καθώς συγκρίνει την απόσταση από κάθε πλευρά. Μέσα το while θα τρέξει n φορές αφού κάθε φορά ταιριάζει μια πλευρά του G' με τις κορυφές του G . Επίσης για το Binary Search χρειάζεστε $O(\log(nk)) = O(\log n + \log k)$ χρόνο.

Άρα συνολικά αυτό το κομμάτι απαιτεί $O(n \log n + n \log n)$

4 Συμπεράσματα

Εάν αθροίσουμε τα επιμέρους κομμάτια παρατηρούμε ότι η συνολική πολυπλοκότητα του αλγορίθμου είναι

$$\begin{aligned} O(kn \log^2(n) + kn \log(n) + nk \log(k) + n \log(n) + n \log(k)) \\ = O(kn(\log^2(n) + \log(k))) \end{aligned}$$

Ένα πρόβλημα που έχει αυτή η προσέγγιση είναι πως, όπως αναφέρθηκε νωρίτερα, η απόσταση Hausdorff θα θέλαμε να είναι ελάχιστη, κάτι που με τυχαία επιλογή για τα c δεν εγγυάται. Επιπλέον για μεγάλες τιμές των c , δε συνεισφέρουν τίποτα στο G . Μία ασφαλής μέθοδος για να τείνει στο 0 θα ήταν να διαλέγουμε μια πυκνότητα και να παίρνουμε ομοιόμορφα τα c , με κάποια προεπιλεγμένη ακτίνα. Όμως μία τέτοια μέθοδος πάλι δε εγγυάται τίποτα για την ταχύτητα σύγκλισης.

References

- [1] Pan, V. Y. (1995). *Optimal (up to polylog factors) sequential and parallel algorithms for approximating complex polynomial zeros*. Proceedings of the twenty-seventh annual ACM symposium on Theory of computing (STOC '95).
- [2] Graham, R.L. (1972). *An Efficient Algorithm for Determining the Convex Hull of a Finite Planar Set*
- [3] O'Rourke, J., Chien, C. B., Olson, T., & Naddor, D. (1982). *A new linear algorithm for intersecting convex polygons*. Computer Graphics and Image Processing, 19(4), 384-391.